

PYTHON HTTP LIBRARY

The **requests** Module

A Complete Technical Reference

requests is a widely used third-party HTTP library for Python that makes sending HTTP requests simple and human-friendly. It wraps the complexity of urllib into a clean API, handling connection pooling, sessions, redirects, encoding, and authentication so developers can focus on working with data rather than protocol details. It is not part of the standard library and must be installed with `pip install requests`.

12 SECTIONS

54 FUNCTIONS COVERED

devforge

01 Making Basic Requests

The core of requests is a set of simple functions, one per HTTP method, that each return a Response object.

`requests.get(url, **kwargs)`

Sends a GET request to retrieve data from a URL. The most commonly used function in the library.

```
import requests
response = requests.get('https://api.github.com')
print(response.status_code)
# 200
```

`requests.post(url, data=None, json=None, **kwargs)`

Sends a POST request, typically used to submit form data or a JSON payload to a server.

```
response = requests.post(
    'https://httpbin.org/post',
    json={'name': 'Rwin'}
)
```

`requests.put(url, data=None, **kwargs)`

Sends a PUT request, typically used to replace an existing resource entirely.

```
response = requests.put(
    'https://api.example.com/items/1',
    json={'title': 'Updated'}
)
```

`requests.patch(url, data=None, **kwargs)`

Sends a PATCH request, used to apply a partial update to an existing resource.

```
response = requests.patch(
    'https://api.example.com/items/1',
    json={'status': 'done'}
)
```

01 Making Basic Requests (cont.)

```
requests.delete(url, **kwargs)
```

Sends a DELETE request, used to remove a resource on the server.

```
response = requests.delete('https://api.example.com/items/1')
```

```
requests.head(url, **kwargs)
```

Sends a HEAD request, identical to GET but without a response body — useful for checking headers or resource existence cheaply.

```
response = requests.head('https://example.com')
print(response.headers['Content-Length'])
```

```
requests.options(url, **kwargs)
```

Sends an OPTIONS request to discover which HTTP methods and headers a server supports for a URL.

```
response = requests.options('https://api.example.com/items')
print(response.headers['Allow'])
```

02 The Response Object

Every request function returns a Response object containing the server's reply, including status, headers, and body content in several formats.

response.status_code

The numeric HTTP status code returned by the server, such as 200, 404, or 500.

```
r = requests.get('https://httpbin.org/status/404')
print(r.status_code)
# 404
```

response.text

The response body decoded as a Unicode string, using the encoding guessed from HTTP headers.

```
r = requests.get('https://example.com')
print(r.text[:100])
```

response.content

The response body as raw bytes, useful for binary content like images or files.

```
r = requests.get('https://example.com/logo.png')
with open('logo.png', 'wb') as f:
    f.write(r.content)
```

response.json()

Parses the response body as JSON and returns it as a Python dict or list. Raises an error if the body is not valid JSON.

```
r = requests.get('https://api.github.com')
data = r.json()
print(data['current_user_url'])
```

02 The Response Object (cont.)

response.headers

A case-insensitive dictionary-like object of the response headers sent by the server.

```
r = requests.get('https://example.com')
print(r.headers['Content-Type'])
```

response.encoding

The character encoding used to decode response.text. Can be set manually if requests guesses incorrectly.

```
r = requests.get('https://example.com')
r.encoding = 'utf-8'
print(r.text)
```

response.url

The final URL of the response, after any redirects have been followed.

```
r = requests.get('http://github.com')
print(r.url)
# https://github.com/
```

response.ok

A boolean shortcut that is True if status_code is less than 400, useful for quick success checks.

```
r = requests.get('https://example.com')
if r.ok:
    print('Success')
```

03 Error Handling & Status Checks

requests distinguishes between connection-level failures and HTTP-level error statuses, giving control over how each is handled.

`response.raise_for_status()`

Raises an `HTTPError` if the status code indicates a client (4xx) or server (5xx) error. Does nothing on success.

```
r = requests.get('https://httpbin.org/status/500')
try:
    r.raise_for_status()
except requests.exceptions.HTTPError as e:
    print('Request failed:', e)
```

`requests.exceptions.ConnectionError`

Raised when a connection to the server cannot be established, such as DNS failure or refused connection.

```
try:
    requests.get('https://nonexistent.invalid')
except requests.exceptions.ConnectionError:
    print('Could not connect')
```

`requests.exceptions.Timeout`

Raised when a request exceeds the time limit set by the `timeout` parameter.

```
try:
    requests.get('https://httpbin.org/delay/5', timeout=2)
except requests.exceptions.Timeout:
    print('Request timed out')
```

`requests.exceptions.TooManyRedirects`

Raised when a request exceeds the maximum number of allowed redirects.

```
try:
    requests.get('https://example.com/loop', allow_redirects=True)
except requests.exceptions.TooManyRedirects:
    print('Redirect loop detected')
```

03 Error Handling & Status Checks (cont.)

`requests.exceptions.RequestException`

The base exception class for all requests errors. Catching this covers connection, timeout, and HTTP errors in one block.

```
try:
    requests.get('https://example.com', timeout=3)
except requests.exceptions.RequestException as e:
    print('Request failed:', e)
```

04 Query Parameters & Headers

requests handles URL encoding and header formatting automatically when passed as Python dictionaries.

params=dict(...)

Passes query string parameters, which requests automatically URL-encodes and appends to the URL.

```
r = requests.get(
    'https://api.github.com/search/repositories',
    params={'q': 'python', 'sort': 'stars'}
)
print(r.url)
```

headers=dict(...)

Sets custom HTTP headers on the request, such as authorization tokens or content type overrides.

```
headers = {'Authorization': 'Bearer TOKEN123'}
r = requests.get('https://api.example.com/me', headers=headers)
```

Custom User-Agent

Overrides the default requests User-Agent string, often required by APIs that block generic clients.

```
headers = {'User-Agent': 'devforge-scanner/1.0'}
r = requests.get('https://api.example.com', headers=headers)
```

response.request.headers

Inspects the headers that were actually sent with the request, useful for debugging authentication issues.

```
r = requests.get('https://example.com', headers={'X-Test': '1'})
print(r.request.headers)
```


05 Sending Data (Forms, JSON, Files)

requests supports multiple ways to send a request body, matching the content type your target API expects.

`data=dict(...)`

Sends data as an application/x-www-form-urlencoded body, mimicking a standard HTML form submission.

```
r = requests.post(
    'https://httpbin.org/post',
    data={'username': 'rwin', 'password': 'secret'}
)
```

`json=dict(...)`

Serializes a Python dict to JSON and sends it as the body, automatically setting Content-Type to application/json.

```
r = requests.post(
    'https://httpbin.org/post',
    json={'event': 'login', 'success': True}
)
```

`files=dict(...)`

Uploads one or more files using multipart/form-data encoding.

```
with open('report.pdf', 'rb') as f:
    r = requests.post(
        'https://httpbin.org/post',
        files={'file': f}
    )
```

`data=generator/bytes`

Streams raw bytes or a generator as the request body directly, useful for large uploads without loading everything into memory.

```
def gen():
    yield b'chunk1'
    yield b'chunk2'

r = requests.post('https://httpbin.org/post', data=gen())
```

06 Sessions

A Session object persists parameters like cookies, headers, and TCP connections across multiple requests, improving both convenience and performance.

`requests.Session()`

Creates a session object that reuses the underlying TCP connection for multiple requests to the same host, significantly improving performance.

```
session = requests.Session()
r1 = session.get('https://api.example.com/login')
r2 = session.get('https://api.example.com/profile')
```

`session.headers.update(dict(...))`

Sets default headers that will be sent with every request made through that session.

```
session = requests.Session()
session.headers.update({'Authorization': 'Bearer TOKEN123'})
r = session.get('https://api.example.com/data')
```

`session.cookies`

A cookie jar that automatically stores cookies received from the server and sends them back on subsequent requests.

```
session = requests.Session()
session.get('https://example.com/set-cookie')
print(session.cookies.get_dict())
```

`with requests.Session() as session:`

Using a session as a context manager ensures the underlying connections are properly closed when finished.

```
with requests.Session() as session:
    r = session.get('https://api.example.com/data')
    print(r.json())
```

07 Authentication

requests provides built-in support for common authentication schemes, and allows custom authentication classes for more complex needs.

auth=(user, pass)

Sends HTTP Basic Authentication credentials with the request as a simple tuple.

```
r = requests.get(
    'https://api.example.com/secure',
    auth=('rwin', 'password123')
)
```

requests.auth.HTTPBasicAuth

The explicit class behind the auth tuple shortcut, useful when you need to construct it separately.

```
from requests.auth import HTTPBasicAuth
r = requests.get(
    'https://api.example.com/secure',
    auth=HTTPBasicAuth('rwin', 'password123')
)
```

requests.auth.HTTPDigestAuth

Implements HTTP Digest Authentication, a more secure alternative to Basic Auth that avoids sending passwords in plaintext.

```
from requests.auth import HTTPDigestAuth
r = requests.get(
    'https://api.example.com/secure',
    auth=HTTPDigestAuth('rwin', 'password123')
)
```

Bearer token via headers

The standard pattern for token-based APIs (OAuth2, JWT) is to pass the token manually in the Authorization header.

```
headers = {'Authorization': 'Bearer eyJhbGciOi...' }
r = requests.get('https://api.example.com/me', headers=headers)
```

08 Timeouts & Retries

By default, requests will wait indefinitely for a response. Setting timeouts and retry logic is essential for production-grade reliability.

`timeout=seconds`

Limits how long requests waits for a connection and response before raising a Timeout exception. Always recommended in production code.

```
r = requests.get('https://api.example.com', timeout=5)
```

`timeout=(connect, read)`

Sets separate timeouts for establishing the connection and for waiting on the response data.

```
r = requests.get(
    'https://api.example.com',
    timeout=(3, 10)
)
```

`requests.adapters.HTTPAdapter`

A transport adapter that can be mounted on a session to configure connection pooling and automatic retries.

```
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

retries = Retry(total=3, backoff_factor=1)
adapter = HTTPAdapter(max_retries=retries)
session = requests.Session()
session.mount('https://', adapter)
```

09 Redirects & Cookies

requests follows redirects automatically for most methods and provides tools to inspect the full redirect chain and any cookies set along the way.

allow_redirects=bool

Controls whether requests automatically follows HTTP redirects. Defaults to True for GET, False for HEAD.

```
r = requests.get(
    'https://example.com/old-page',
    allow_redirects=False
)
print(r.status_code)
# 301
```

response.history

A list of Response objects representing each redirect that was followed before reaching the final URL.

```
r = requests.get('http://github.com')
for step in r.history:
    print(step.status_code, step.url)
```

requests.utils.dict_from_cookiejar

Converts a cookie jar (such as response.cookies) into a plain Python dictionary.

```
r = requests.get('https://example.com')
cookies = requests.utils.dict_from_cookiejar(r.cookies)
print(cookies)
```

cookies=dict(...)

Sends custom cookies with a request, without needing a full session.

```
r = requests.get(
    'https://example.com',
    cookies={'session_id': 'abc123'}
)
```

10 SSL Verification & Proxies

requests verifies SSL certificates by default and supports routing traffic through proxies, both common concerns in security-focused tooling.

verify=True/False

Controls whether requests verifies the server's SSL certificate. Disabling this is insecure and should only be used for trusted testing environments.

```
r = requests.get('https://self-signed.example.com', verify=False)
```

verify='path/to/cert.pem'

Points requests to a custom CA bundle, useful for internal services with private certificate authorities.

```
r = requests.get(
    'https://internal.example.com',
    verify='/etc/ssl/internal-ca.pem'
)
```

proxies=dict(...)

Routes the request through an HTTP or SOCKS proxy, specified per protocol scheme.

```
proxies = {
    'http': 'http://10.0.0.1:8080',
    'https': 'http://10.0.0.1:8080',
}
r = requests.get('https://example.com', proxies=proxies)
```

cert=('cert.pem', 'key.pem')

Presents a client-side SSL certificate for mutual TLS authentication with the server.

```
r = requests.get(
    'https://mtls.example.com',
    cert=('client.pem', 'client-key.pem')
)
```

11 Streaming Responses

For large downloads, requests can stream the response body incrementally instead of loading it entirely into memory.

`stream=True`

Delays downloading the response body until it is explicitly accessed, enabling chunked processing.

```
r = requests.get('https://example.com/bigfile.zip', stream=True)
print(r.status_code)
# Body not yet downloaded
```

`response.iter_content(chunk_size)`

Iterates over the response body in fixed-size chunks, ideal for writing large files to disk without high memory use.

```
r = requests.get('https://example.com/bigfile.zip', stream=True)
with open('bigfile.zip', 'wb') as f:
    for chunk in r.iter_content(chunk_size=8192):
        f.write(chunk)
```

`response.iter_lines()`

Iterates over the response body line by line, useful for streaming text-based formats like NDJSON or server logs.

```
r = requests.get('https://example.com/logs', stream=True)
for line in r.iter_lines():
    if line:
        print(line.decode('utf-8'))
```

`response.raw`

Gives access to the raw urllib3 HTTPResponse object for advanced use cases like reading with a custom buffer size.

```
r = requests.get('https://example.com/bigfile.zip', stream=True)
data = r.raw.read(1024)
```

12 The PreparedRequest Object

Internally, every call builds a PreparedRequest before sending it. Accessing this directly enables fine-grained control and inspection.

`requests.Request(...)`

Constructs a Request object manually, which can then be prepared and sent through a session.

```
req = requests.Request(
    'POST', 'https://api.example.com/data',
    json={'key': 'value'}
)
```

`session.prepare_request(request)`

Converts a Request object into a PreparedRequest, merging in session-level settings like headers and cookies.

```
session = requests.Session()
prepared = session.prepare_request(req)
print(prepared.url)
print(prepared.body)
```

`session.send(prepared_request)`

Sends a previously prepared request, allowing inspection or modification before it goes out on the wire.

```
response = session.send(prepared)
print(response.status_code)
```


Best Practices & Recommendations

Practical guidance for using the requests module safely and idiomatically.

- 01 Always set a timeout on every request; a hanging connection with no timeout can freeze an entire script indefinitely.
- 02 Use a Session object when making multiple requests to the same host — it reuses TCP connections and is noticeably faster.
- 03 Prefer `response.raise_for_status()` over manually checking `status_code` for every possible error case.
- 04 Never disable SSL verification (`verify=False`) in production code; use it only for local testing against trusted hosts.
- 05 Use the `json` parameter instead of manually calling `json.dumps()` and setting headers yourself — requests handles both automatically.
- 06 For large file downloads, always use `stream=True` with `iter_content()` to avoid loading the entire file into memory at once.